

Ramas y Deploy

Módulo: Desarrollo de Interfaces (0488)

Contexto Curricular:

- **RA 7:** Distribución y empaquetado de aplicaciones.
- **CE 7.c:** Se han utilizado herramientas de control de versiones.
- **CE 7.g:** Se han documentado las fases del proceso de distribución y despliegue continuo (CI/CD).

1. Base Teórica Técnica: VCS, Ramas y Grafo Dirigido Acíclico (DAG)

Git es un gestor de estados basado en un **Grafo Dirigido Acíclico (DAG)**.

- **El Grafo (DAG):** Cada vez que realizas un *commit*, Git genera un objeto inmutable que contiene un puntero a una captura (el *tree* de tu proyecto), metadatos (autor, mensaje) y punteros a los *commits* padre. Esto forma un grafo donde la información fluye en una dirección y nunca forma bucles.
- **Las Ramas (Branches):** En Git, una rama no es una carpeta aislada, es simplemente un archivo de 41 bytes que contiene un apuntador (el hash SHA-1) hacia un nodo específico del DAG. Cuando creas la rama desarrollo, simplemente estás creando un nuevo puntero que apunta al mismo *commit* actual. Al añadir nuevos *commits*, el puntero de la rama avanza, mientras que los demás se quedan atrás.
- **Los Remotos (git remote):** Son alias semánticos (como origin) que apuntan a la URL de otro repositorio (por ejemplo, en GitHub). El comando `git remote add` no mueve archivos; simplemente registra esa URL en el archivo de configuración `.git/config` para que operaciones como `push` o `fetch` sepan a qué servidor transferir los objetos del DAG que faltan.

git config - - list

2. Flujo de Trabajo Técnico: Clonación, Bifurcación y Despliegue en Vercel

1. Clonar el repositorio base:

Clona el código fuente en tu máquina local. Esto descarga el historial .git y extrae el árbol de trabajo actual.

CTRL + Shift + P

```
git clone https://github.com/usuario-origen/repo-base.git
cd repo-base
```

2. Desvincular y reasignar el Remoto (Git Remote): Requisito: Haber creado un repositorio vacío en tu GitHub.

Por defecto, Git asigna el nombre origin a la URL de la que clonaste. Si intentas hacer push ahora, te dará un error 403 (Permiso denegado) porque no eres el dueño del repo original. Debes eliminar ese enlace y apuntar a tu nuevo repositorio público.

```
Bash
# Eliminar el enlace al repositorio original
git remote remove origin

# Añadir tu nuevo repositorio como el destino 'origin'
git remote add origin https://github.com/carmelo-escribano/mi-nuevo-repo.git
```

3. Creación de la rama 'desarrollo' y modificaciones:

Creamos el nuevo puntero (rama) y nos movemos a él simultáneamente con el flag -b.

```
Bash
git checkout -b desarrollo

# Realiza aquí los cambios en el código usando VS Code...

git add .
git commit -m "feat: modificaciones iniciales para entorno de desarrollo"
```

4.Subir el grafo al Remoto (Push):

El flag -u (upstream) es crucial en este primer push. Le indica a Git local que vincule la rama desarrollo local con la rama desarrollo que está a punto de crear en GitHub, facilitando futuros git pull o git push sin necesidad de especificar la rama.

```
Bash
git push -u origin desarrollo
```

5.Configurar 'desarrollo' como rama por defecto en GitHub:

GitHub establece main (o master) como la rama por defecto al visualizar el código. Para cambiar esto:

1. Ve a tu repositorio en GitHub y haz clic en la pestaña **Settings**.
2. En el menú lateral izquierdo, selecciona **Default branch** (bajo *Code and automation*).
3. Haz clic en el icono de las dos flechas (*Switch default branch*).
4. Selecciona desarrollo y haz clic en **Update**. Entiende la advertencia de GitHub (afecta a PRs base) y confirma.

6.Despliegue y Configuración en Vercel:Sobrescribir la rama de producción.

Por defecto, Vercel despliega a producción cualquier commit que entre en la rama por defecto de GitHub. Si necesitas forzar que Vercel trate a desarrollo como la rama de producción:

1. Ve al dashboard de Vercel y haz clic en **Add New -> Project**.
2. Importa tu repositorio desde GitHub.
3. Antes de darle a *Deploy*, despliega la sección **Build and Output Settings** o ve directo a las configuraciones del proyecto (si ya lo importaste).
4. En Vercel: **Settings -> Git -> Production Branch**.
5. Escribe desarrollo en el campo y guarda. A partir de este momento, cada push a la rama desarrollo disparará una *build* de producción, y las demás ramas generarán *Preview Deployments*.